



UNIVERSITI  
TEKNOLOGI  
MARA

Cawangan Terengganu  
Kampus Kuala Terengganu

**FACULTY OF COMPUTER AND MATHEMATICAL SCIENCES**

**CSC580**

**PARALLEL PROCESSING**

**SEMESTER 6 (2026)**

**GROUP PROJECT:**

**DISTRIBUTED DATA ANALYTICS USING MESSAGE PASSING  
INTERFACE (MPI)**

**PREPARED BY:**

| <b>STUDENT ID</b> | <b>NAME</b>                      | <b>GROUP</b> |
|-------------------|----------------------------------|--------------|
| 2023889726        | MUHAMMAD AMINUDDIN BIN HASNAN    | CDCS2306B1   |
| 2023213904        | NUR HAJARIATUL ASMA BINTI SHAARI | CDCS2306B1   |
| 2024204526        | SITI NURAINA BINTI MAHADI        | CDCS2306B1   |
| 2023414904        | ZUHAIRI IKHWAN BIN ZUBIR         | CDCS2306B1   |

**PREPARED FOR:**

**MUHAMMAD ATIF BIN RAMLAN**

**DATE OF SUBMISSION:**

**4 JULY 2026**

# TABLE OF CONTENTS

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>ABSTRACT .....</b>                                             | <b>4</b>  |
| <b>1.0 INTRODUCTION .....</b>                                     | <b>5</b>  |
| <b>1.1 Background .....</b>                                       | <b>5</b>  |
| <b>1.2 Motivation .....</b>                                       | <b>5</b>  |
| <b>1.3 Problem statement .....</b>                                | <b>6</b>  |
| <b>1.4 Objectives .....</b>                                       | <b>6</b>  |
| <b>2.0 SYSTEM SETUP.....</b>                                      | <b>6</b>  |
| <b>2.1 Hardware specifications .....</b>                          | <b>6</b>  |
| <b>2.2 Network configuration .....</b>                            | <b>7</b>  |
| <b>2.3 MPI installation steps.....</b>                            | <b>7</b>  |
| <b>3.0 ALGORITHM DESIGN .....</b>                                 | <b>8</b>  |
| <b>3.1 Sequential algorithm design .....</b>                      | <b>8</b>  |
| <b>3.1.1 Sequential Pseudocode .....</b>                          | <b>8</b>  |
| <b>3.1.2 Sequential Flowcharts.....</b>                           | <b>10</b> |
| <b>3.2 MPI algorithm design.....</b>                              | <b>10</b> |
| <b>3.2.1 MPI Pseudocode .....</b>                                 | <b>11</b> |
| <b>3.2.2 MPI Flowcharts .....</b>                                 | <b>14</b> |
| <b>4.0 IMPLEMENTATION .....</b>                                   | <b>14</b> |
| <b>4.1 Sequential Implementation.....</b>                         | <b>14</b> |
| <b>4.2 MPI Implementation.....</b>                                | <b>15</b> |
| <b>4.3 Basic Statistic.....</b>                                   | <b>15</b> |
| <b>4.4 Histogram Generation.....</b>                              | <b>16</b> |
| <b>4.5 Sorting .....</b>                                          | <b>16</b> |
| <b>4.6 Pearson Correlation .....</b>                              | <b>17</b> |
| <b>4.7 Moving Average .....</b>                                   | <b>17</b> |
| <b>4.8 Outlier Detection.....</b>                                 | <b>17</b> |
| <b>5.0 EXPERIMENTAL SETUP.....</b>                                | <b>18</b> |
| <b>5.1 Dataset Description and Experiment Configuration .....</b> | <b>18</b> |
| <b>5.2 Performance Metrics.....</b>                               | <b>19</b> |
| <b>6.0 RESULTS AND DISCUSSION.....</b>                            | <b>20</b> |
| <b>6.1 Runtime Result Table .....</b>                             | <b>20</b> |
| <b>6.2 Runtime Analysis .....</b>                                 | <b>21</b> |
| <b>6.3 Performance of Individual Analytical Tasks .....</b>       | <b>21</b> |
| <b>6.4 Distributed MPI Performance .....</b>                      | <b>22</b> |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>6.5 Discussion of Findings.....</b>          | <b>23</b> |
| <b>7.0 CHALLENGES AND LESSONS LEARNED .....</b> | <b>23</b> |
| <b>8.0 CONCLUSION .....</b>                     | <b>25</b> |
| <b>9.0 INDIVIDUAL CONTRIBUTION TABLE .....</b>  | <b>25</b> |
| <b>10.0 REFERENCES.....</b>                     | <b>26</b> |

## ABSTRACT

This project presents a performance comparison between a sequential statistical data analytics pipeline and a parallel implementation using Message Passing Interface (MPI). The purpose of this project is to evaluate how MPI can improve runtime performance when processing large datasets. The system performs several analytical tasks, including basic statistics, histogram generation, sorting, Pearson correlation, moving average calculation, and outlier detection. A small sales dataset was prepared to represent the original data structure, while larger synthetic numeric datasets were generated in the program to test scalability. The sequential version processes the full dataset using one process only, while the MPI version divides the workload among multiple processes. MPI functions such as `MPI_Bcast`, `MPI_Scatterv`, `MPI_Reduce`, and `MPI_Gatherv` were used to distribute data and combine partial results. The experiment was conducted using dataset sizes of 100,000, 500,000, 1,000,000, 5,000,000, and 10,000,000 records. The result shows that MPI is slower for the smallest dataset because communication overhead is higher than the actual processing workload. However, MPI becomes faster as the dataset size increases. The best result was achieved at 5,000,000 records, where MPI obtained 56.65% runtime improvement, 2.31 speedup, and 57.67% parallel efficiency. A distributed MPI test using three laptops was also conducted to show real network-based MPI execution. Overall, MPI is suitable for medium and large datasets, but careful network setup and memory management are required.

## **1.0 INTRODUCTION**

### **1.1 Background**

Parallel computing is a method for working with a large problem by breaking it into smaller chunks, which allows the smaller chunks to be performed simultaneously. It is useful in the field of data analytics because of the large number of times you need to perform the same operation against a large data set, such as summation, sorting, histogram counting, moving averages and statistical analysis. In a small data set, sequential processing may be sufficient; however, as the size of a data set grows, the time it may take to process a large data set using only sequential processing can grow significantly because only one process does all the work. The Message Passing Interface (MPI) is a parallel programming model that allows multiple processes to communicate with one another. MPI is widely used in high performance computing to allow a program to take a larger data set, break it into smaller segments, and process them concurrently. In this project, MPI is used to improve the performance of a program that uses statistical analytics to analysed data.

### **1.2 Motivation**

This project aims to evaluate how well MPI parallel processing performs compared to traditional serial processing. A real-world application for processing large databases is to get results back from your analysis in as short as possible a time. The longer it takes to complete a task, the longer it takes to improve your analysis tool processes. As such, this project will look at how well MPI can decrease the time it takes to perform actions on large amounts of data as an increasing amount of data is being processed simultaneously. Additionally, this project provides insight into the benefits and drawbacks of using MPI. By using multiple processes, MPI has been shown to improve execution times; however, the increased number of processes creates added overhead caused by data distribution, inter-process communication, coordination, and results collection. Therefore, MPI will not always outperform single-threaded execution, especially with smaller datasets.

### 1.3 Problem statement

It's easy to build sequential programs as they perform instructions in a sequential manner with a single unit of processing. Unfortunately, when the number of items in the data set increases, sequential processing also increases the computation time since each instruction is executed by a single processing unit all instructions are executed by the same processor. The Method of Parallelism Interface (MPI) resolves this problem by dividing the computations among multiple processors. However, using more than one processor increases the cost of communication. Thus, the primary concern with this project is whether implementing MPI can improve runtime performance over sequential implementation at various data set lengths.

### 1.4 Objectives

1. To develop a sequential statistical data analytics program as the baseline implementation.
2. To develop an MPI-based parallel statistical data analytics program.
3. To compare the performance of sequential and MPI implementations using runtime, speedup, improvement percentage, and parallel efficiency.
4. To analyse the effect of different dataset sizes on MPI performance.
5. To test MPI execution in both local and distributed laptop network environments.

## 2.0 SYSTEM SETUP

### 2.1 Hardware specifications

The experiment was conducted using Windows laptops. Based on the ipconfig screenshots, three laptops were successfully connected to the same Wi-Fi network for the distributed MPI test. The CPU, RAM, and storage should be filled in based on the actual laptops used.

| Device   | Operating System / Windows Version | Processor     | RAM           | Storage           | Role              |
|----------|------------------------------------|---------------|---------------|-------------------|-------------------|
| Laptop 1 | Windows Version 10.0.26200.8328    | [Fill in CPU] | [Fill in RAM] | [Fill in Storage] | MPI node / worker |
| Laptop 2 | Windows Version 10.0.26200.8737    | [Fill in CPU] | [Fill in RAM] | [Fill in Storage] | MPI node / worker |

|          |                                       |               |               |                      |                      |
|----------|---------------------------------------|---------------|---------------|----------------------|----------------------|
| Laptop 3 | Windows<br>Version<br>10.0.26200.8655 | [Fill in CPU] | [Fill in RAM] | [Fill in<br>Storage] | MPI node /<br>worker |
|----------|---------------------------------------|---------------|---------------|----------------------|----------------------|

## 2.2 Network configuration

For the distributed MPI experiment, the group attempted to connect four laptops. However, one laptop could not connect because it was using a different router or network. Therefore, only three laptops were successfully connected and used for the distributed MPI test.

Based on the ipconfig result, the three connected laptops used the following Wi-Fi network configuration:

| Device   | Wi-Fi IPv4<br>Address | Subnet Mask   | Default Gateway | Status    |
|----------|-----------------------|---------------|-----------------|-----------|
| Laptop 1 | 10.62.52.110          | 255.255.240.0 | 10.62.63.254    | Connected |
| Laptop 2 | 10.62.53.182          | 255.255.240.0 | 10.62.63.254    | Connected |
| Laptop 3 | 10.62.50.208          | 255.255.240.0 | 10.62.63.254    | Connected |

The IP addresses show that all three laptops were connected under the same network range, which is 10.62.x.x. They also shared the same subnet mask and default gateway. This means the laptops were able to communicate within the same network, which is required for MPI execution across multiple devices.

Other adapters such as vEthernet, VMware Network Adapter, Cloudflare WARP, and Bluetooth Network Connection were not used because they are virtual or disconnected adapters. The important adapter for this experiment was the Wireless LAN adapter Wi-Fi.

## 2.3 MPI installation steps

|    |                                                                                |
|----|--------------------------------------------------------------------------------|
| 1. | Install Microsoft MPI Runtime.                                                 |
| 2. | Install Microsoft MPI SDK.                                                     |
| 3. | Install Microsoft Visual Studio or Visual Studio Build Tools with C++ support. |

|    |                                                         |
|----|---------------------------------------------------------|
| 4. | Open x64 Native Tools Command Prompt for Visual Studio. |
| 5. | Compile the sequential C++ program.                     |
| 6. | Compile the MPI C++ program by linking the MPI library. |
| 7. | Run the sequential program normally.                    |
| 8. | Run the MPI program using mpiexec.                      |

Distributed MPI run using three laptops:

```
C:\Windows\System32>mpiexec -hosts 3 192.168.1.101 1 192.168.1.102 1 192.168.1.100 1 hostname
LAPTOP-GKDHQ114
DESKTOP-D40HQPR
LAPTOP-PKBNKEOI
```

## 3.0 ALGORITHM DESIGN

### 3.1 Sequential algorithm design

In a linear process, the sequential algorithm works on an entire dataset using just one process. All the data analytics tasks are done in a linear fashion each task is performed one after another. The program first generates the full dataset, then calculates statistics, generates histograms, sorts the data, calculates Pearson correlations, calculates moving averages and detects outliers. Once all data analytics tasks have been completed, the total time taken will be recorded in the results file.

#### 3.1.1 Sequential Pseudocode

```
START

Input dataset size N

Generate random dataset data1
Generate second random dataset data2

Start timer

Calculate basic statistics:
    sum = 0
    FOR each value in data1
        sum = sum + value
    END FOR

    mean = sum / N

Calculate variance:
    variance_sum = 0
    FOR each value in data1
        variance_sum = variance_sum + (value - mean)^2
```

```

        END FOR

        variance = variance_sum / N
        standard_deviation = sqrt(variance)

Find minimum value in data1
Find maximum value in data1

Generate histogram:
    Create 10 bins
    FOR each value in data1
        Find suitable bin
        Increase bin count
    END FOR

Sort dataset:
    Sort all values in data1
    Find smallest value
    Find largest value
    Find median value

Calculate Pearson correlation:
    Calculate sumX, sumY, sumXY, sumX2, sumY2
    Apply Pearson correlation formula

Calculate moving average:
    Set window size = 5
    Calculate first moving average
    Calculate last moving average

Detect outliers:
    lower_bound = mean - 3 * standard_deviation
    upper_bound = mean + 3 * standard_deviation

    FOR each value in data1
        IF value < lower_bound OR value > upper_bound
            Count as outlier
        END IF
    END FOR

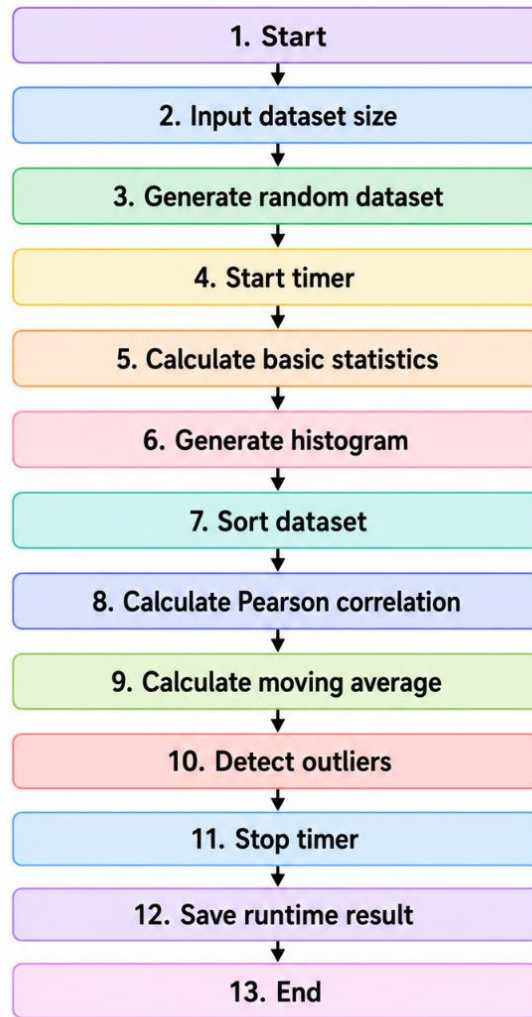
Stop timer

Display results
Save runtime into sequential_results.csv

END

```

### 3.1.2 Sequential Flowcharts



### 3.2 MPI algorithm design

The MPI algorithm in this project uses multiple processes to perform statistical data analytics in parallel. The program begins by calling `MPI_Init` to start the MPI environment. Each process then gets its rank and the total number of processes using `MPI_Comm_rank` and `MPI_Comm_size`.

Rank 0 acts as the main process. It receives the dataset size and generates the full dataset using random double values from 0 to 10000. Rank 0 also generates a second dataset that is partially related to the first dataset. This second dataset is used for Pearson correlation calculation. After the dataset is generated, the dataset size is shared with all processes using `MPI_Bcast`. The dataset is then divided into chunks using `sendCounts` and `displacements`. These values make sure the dataset

can be divided correctly even if the dataset size is not exactly divisible by the number of processes.

The data chunks are distributed to all processes using `MPI_Scatterv`. Each process performs analytics on its own local data chunk. Partial results such as local sum, local minimum, local maximum, histogram count, Pearson correlation values, and outlier count are combined using `MPI_Reduce`. For sorting, each process sorts its local data first. Then, all sorted chunks are gathered back to Rank 0 using `MPI_Gatherv`. Rank 0 performs the final global sorting and calculates the smallest value, largest value, and median. Finally, Rank 0 displays the results and saves the timing result into `mpi_results.csv` inside the project results folder.

### 3.2.1 MPI Pseudocode

```
START

MPI_Init

Get process rank using MPI_Comm_rank
Get total number of processes using MPI_Comm_size

Set default dataset size N = 1,000,000

IF rank == 0 THEN
  IF user enters dataset size
    N = user input dataset size
  END IF

  Display program title
  Display dataset size
  Display number of processes
END IF

Broadcast dataset size N to all processes using MPI_Bcast

Calculate base chunk size:
base = N / number of processes
remainder = N % number of processes

Create sendCounts and displacements:
FOR each process
  Assign chunk size
  Add extra record if process rank is less than remainder
END FOR

IF rank == 0 THEN
  Generate main dataset using random values from 0 to 10000
```

```

Generate second dataset for Pearson correlation
END IF

Scatter main dataset to all processes using MPI_Scatterv
Scatter second dataset to all processes using MPI_Scatterv

Create original copy of local data

Start total timer

1. Basic Statistics:
Each process calculates:
local sum
local minimum
local maximum

Use MPI_Reduce to combine:
global sum
global minimum
global maximum

Rank 0 calculates mean

Broadcast mean to all processes using MPI_Bcast

Each process calculates local variance sum

Use MPI_Reduce to combine global variance sum

Rank 0 calculates variance and standard deviation

Broadcast variance and standard deviation to all processes

2. Histogram:
Each process creates local histogram with 10 bins

FOR each local value
Determine bin number
Increase local histogram count
END FOR

Use MPI_Reduce to combine local histograms into global histogram

3. Sorting:
Each process sorts local data chunk

Use MPI_Gatherv to gather sorted chunks to Rank 0

IF rank == 0 THEN
Sort gathered data globally
Find smallest value
Find largest value
Find median value
END IF

```

```

4. Pearson Correlation:
Each process calculates:
localSumX
localSumY
localSumXY
localSumX2
localSumY2

Use MPI_Reduce to combine all partial sums

IF rank == 0 THEN
Calculate Pearson correlation value
END IF

5. Moving Average:
IF rank == 0 THEN
Use window size = 5
Calculate first moving average
Calculate last moving average
END IF

6. Outlier Detection:
Calculate lower bound:
mean - 3 * standard deviation

Calculate upper bound:
mean + 3 * standard deviation

Each process counts local outliers

Use MPI_Reduce to combine total outlier count

Stop total timer

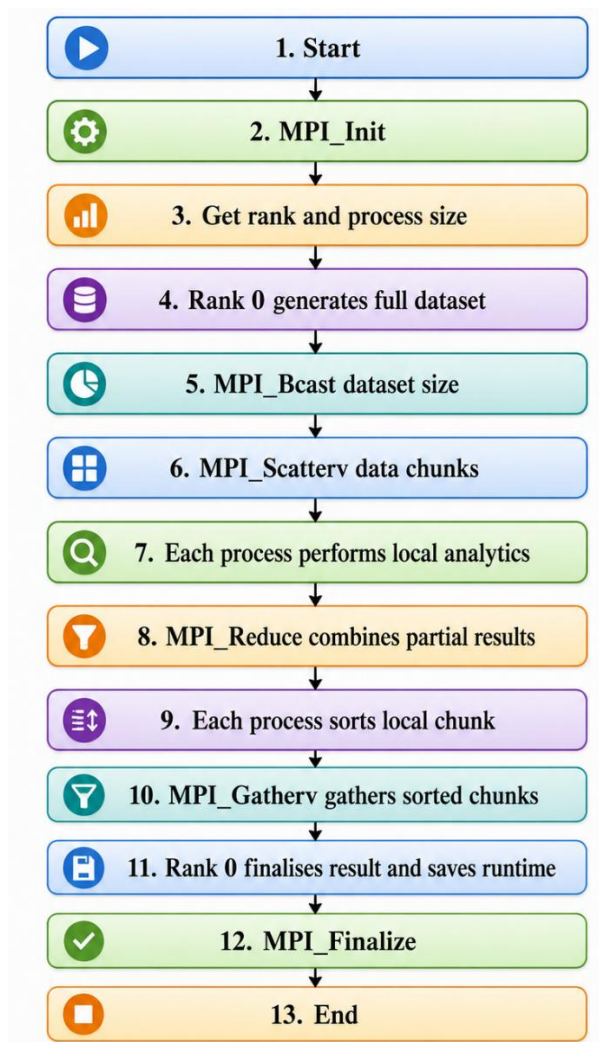
IF rank == 0 THEN
Display final analytical results
Display timing result for each task
Save timing result into mpi_results.csv
END IF

MPI_Finalize

END

```

### 3.2.2 MPI Flowcharts



## 4.0 IMPLEMENTATION

### 4.1 Sequential Implementation

The sequential implementation is written in `sequential_analytics.cpp`. This program acts as the baseline version. It runs all analytics tasks using only one process. The runtime result is saved into `sequential_results.csv`.

The sequential program follows this order:

1. Generate Dataset
2. Calculate basic statistics
3. Generate histogram
4. Sort dataset
5. Calculate Pearson correlation

6. Calculate moving average
7. Detect outliers
8. Save runtime result

## 4.2 MPI Implementation

The MPI implementation is written in `mpi_analytics.cpp`. It improves the sequential version by dividing the workload among multiple processes. Rank 0 generates the dataset and distributes the data to other processes. Each process performs calculations on its own chunk, and the final result is combined using MPI communication functions.

Important MPI functions used:

| MPI Function               | Purpose                                              |
|----------------------------|------------------------------------------------------|
| <code>MPI_Init</code>      | Starts the MPI environment                           |
| <code>MPI_Comm_rank</code> | Gets the process rank                                |
| <code>MPI_Comm_size</code> | Gets total number of processes                       |
| <code>MPI_Bcast</code>     | Sends dataset size or global values to all processes |
| <code>MPI_Scatterv</code>  | Distributes different data chunks to processes       |
| <code>MPI_Reduce</code>    | Combines partial results into one final result       |
| <code>MPI_Gatherv</code>   | Gathers data chunks back to Rank 0                   |
| <code>MPI_Barrier</code>   | Synchronizes all processes before timing each task   |
| <code>MPI_Finalize</code>  | Ends the MPI environment                             |

## 4.3 Basic Statistic

Basic statistics include mean, variance, standard deviation, minimum, and maximum.

Example sequential code:

```
double sum = accumulate(data.begin(), data.end(), 0.0);
double mean = sum / data.size();

double varianceSum = 0.0;
for (double value : data) {
    varianceSum += (value - mean) * (value - mean);
}

double variance = varianceSum / data.size();
double stddev = sqrt(variance);
double minimum = *min_element(data.begin(), data.end());
double maximum = *max_element(data.begin(), data.end());
```

In the MPI implementation, each process calculates its local sum, local minimum, and local maximum. Then, `MPI_Reduce` is used to combine the partial results into global results.

## 4.4 Histogram Generation

The histogram task divides the generated values into 10 bins. Since the data values are generated between 0 and 10000, each bin represents a range of 1000.

Example MPI logic:

```
const int bins = 10;
int bin = static_cast<int>(x / 1000.0);

if (bin < 0) bin = 0;
if (bin >= bins) bin = bins - 1;

localHist[bin]++;

MPI_Reduce(localHist, globalHist, bins, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);
```

Each process creates a local histogram using its own data chunk. Then, all local histograms are combined into a global histogram using `MPI_Reduce`.

## 4.5 Sorting

Sorting is used to arrange the dataset and find values such as the smallest value, largest value, and median.

Example MPI sorting logic:

```
vector<double> sortedLocalData = originalLocalData;

sort(sortedLocalData.begin(), sortedLocalData.end());

MPI_Gatherv(sortedLocalData.data(), localCount, MPI_DOUBLE,
gatheredData.data(), sendCounts.data(), displacements.data(),
MPI_DOUBLE, 0, MPI_COMM_WORLD);

if (rank == 0) {
    sort(gatheredData.begin(), gatheredData.end());
}
```

Each process sorts its own local data first. After that, the sorted chunks are gathered back to Rank 0. Rank 0 performs the final global sort. This method works, but it can create a bottleneck because Rank 0 still needs to handle the final sorting.

## 4.6 Pearson Correlation

Pearson correlation is used to measure the relationship between two datasets.

Example MPI logic:

```
localSumX += localData1[i];
localSumY += localData2[i];
localSumXY += localData1[i] * localData2[i];
localSumX2 += localData1[i] * localData1[i];
localSumY2 += localData2[i] * localData2[i];

MPI_Reduce(&localSumXY, &globalSumXY, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
```

Each process calculates partial sums for its local chunk. Then, `MPI_Reduce` combines the values. Rank 0 then calculates the final Pearson correlation.

## 4.7 Moving Average

The moving average task uses a window size of 5. It is used to smooth the data by calculating the average of nearby values.

Example code:

```
const int window = 5;

for (int i = 0; i < window; i++)
    firstSum += fullData[i];

for (long long i = N - window; i < N; i++)
    lastSum += fullData[i];

firstMovingAverage = firstSum / window;
lastMovingAverage = lastSum / window;
```

The moving average is useful to observe the general trend of values. In this project, the first and last moving average values are calculated.

## 4.8 Outlier Detection

Outlier detection is used to identify values that are very far from the mean. The method used is mean plus or minus three standard deviations.

Example MPI logic:

```
double lowerBound = mean - (3.0 * stddev);
double upperBound = mean + (3.0 * stddev);

for (double x : originalLocalData) {
    if (x < lowerBound || x > upperBound)
```

```

    localOutliers++;
}

MPI_Reduce(&localOutliers, &globalOutliers, 1, MPI_LONG_LONG, MPI_SUM, 0,
MPI_COMM_WORLD);

```

Each process counts the number of outliers in its local data. Then, all local outlier counts are combined using `MPI_Reduce`.

## 5.0 EXPERIMENTAL SETUP

### 5.1 Dataset Description and Experiment Configuration

This project employs a sales dataset called `sales.csv`, which consists of 4 columns with the following columns: Region, Product, Quantity and Price. The Region column identifies where those sales occurred across four geographic regions: North, South, East and West. In addition, the Product column outlines the type of product being sold in this case Laptops, Phones, Tablets and Monitors. The Quantity column tracks how many units were sold, and the Price column lists the price charged for each of the products. The quantity and price numeric data will be used to conduct statistical analyses for the project. As an example, the two numeric datasets will allow for the following statistical calculations to be performed: total quantity; total dollar amount of sales; average sales; minimum sale; maximum sale; and other various types of statistical functions.

Table x.x : sample records from `sales.csv`

| Region | Product | Quantity | Price |
|--------|---------|----------|-------|
| North  | Laptop  | 12       | 3500  |
| North  | Phone   | 25       | 1800  |
| South  | Laptop  | 8        | 3500  |
| South  | Phone   | 30       | 1800  |
| East   | Tablet  | 12       | 1200  |
| West   | Monitor | 10       | 800   |

The experiment was configured to compare the sequential C++ implementation with the MPI-based parallel implementation. In the sequential version, the whole dataset is processed using one process. In the MPI version, the dataset is divided into smaller chunks and distributed to multiple MPI processes. Each process performs calculations

on its assigned data records, and the partial results are then combined by the master process to produce the result. This experiment is used to observe how MPI distributes workload and how the performance differs from sequential processing. The performance is evaluated using execution time, speedup, efficiency, and parallel overhead.

Table x.x : Simple Table for Experiment Configuration

| Configuration item    | Value used                                         |
|-----------------------|----------------------------------------------------|
| Programming language  | C++                                                |
| Parallel library      | Microsoft MPI                                      |
| Default dataset size  | 1,000,000 records                                  |
| Tested dataset sizes  | 100,000; 500,000; 1,000,000; 5,000,000; 10,000,000 |
| Data type             | Random double values                               |
| Data range            | 0 to 10000                                         |
| First dataset seed    | 42                                                 |
| Second dataset seed   | 99                                                 |
| Histogram bins        | 10                                                 |
| Moving average window | 5                                                  |
| Local MPI processes   | 4 processes                                        |
| Distributed MPI setup | 3 laptops                                          |
| Runtime unit          | Milliseconds, ms                                   |
| Output file           | mpi_results.csv                                    |

Two types of MPI testing were performed:

| Experiment Type         | Description                                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| Local MPI testing       | MPI was run on one computer using 4 processes. This result was used for the main performance charts.         |
| Distributed MPI testing | MPI was run using 3 connected laptops on the same network. This was used to test real MPI network execution. |

## 5.2 Performance Metrics

The performance of the sequential and MPI programs was compared using four main metrics: runtime, speedup, improvement percentage, and parallel efficiency. These

metrics help to show whether the MPI program performs better than the sequential program when the dataset size increases.

Runtime refers to the total time taken by a program to complete all analytical tasks. In this project, runtime is measured in milliseconds. A lower runtime means the program performs faster. By comparing the runtime of the sequential program and the MPI program, we can identify which implementation is more efficient for each dataset size.

Speedup is used to measure how many times faster the MPI program is compared to the sequential program. It is calculated by dividing the sequential runtime by the MPI runtime. If the speedup value is greater than 1, it means MPI is faster than the sequential program. However, if the value is less than 1, it means the sequential program is faster.

Improvement percentage shows how much runtime is reduced when using MPI compared to the sequential implementation. It is calculated by finding the difference between sequential runtime and MPI runtime, then dividing it by the sequential runtime and multiplying by 100. A positive improvement percentage means MPI improves the performance, while a negative percentage means MPI is slower.

Parallel efficiency measures how well the MPI processes are being used. It is calculated by dividing the speedup by the number of MPI processes and multiplying by 100. Higher efficiency means the MPI processes are used effectively. However, the efficiency may decrease if there is communication overhead, workload imbalance, or extra time needed to gather results from different processes.

## 6.0 RESULTS AND DISCUSSION

### 6.1 Runtime Result Table

Table 8.1 presents the performance results obtained from the MPI statistical analytics program using dataset sizes of **1,000,000** and **10,000,000** records. The runtime is divided into six analytical tasks, namely Basic Statistics, Histogram Generation, Sorting, Pearson Correlation, Moving Average, and Outlier Detection.

| Dat<br>aset<br>Size | Pro<br>cese<br>s | Total<br>_ms  | BasicSta<br>tistics_m<br>s | Histog<br>ram_m<br>s | Sorti<br>ng_m<br>s | PearsonCo<br>rrelation_<br>ms | MovingA<br>verage_m<br>s | OutlierDe<br>tection_m<br>s |
|---------------------|------------------|---------------|----------------------------|----------------------|--------------------|-------------------------------|--------------------------|-----------------------------|
| <b>1000<br/>000</b> | <b>3</b>         | 8331.<br>9176 | 100.7042                   | 11.780<br>5          | 3038.<br>6919      | 19.7491                       | 3.8035                   | 11.9438                     |

|                      |   |                    |          |             |                    |          |       |         |
|----------------------|---|--------------------|----------|-------------|--------------------|----------|-------|---------|
| <b>1000<br/>0000</b> | 3 | 7856<br>5.488<br>7 | 119.7792 | 30.707<br>6 | 3821<br>2.524<br>5 | 161.9275 | 3.228 | 89.8365 |
|----------------------|---|--------------------|----------|-------------|--------------------|----------|-------|---------|

## 6.2 Runtime Analysis

The experiment focused on evaluating the execution time of the MPI statistical analytics program using large datasets containing **1,000,000** and **10,000,000** records. The results demonstrate that the total runtime increases significantly as the dataset size becomes larger.

For the dataset size of **1,000,000 records**, the program completed execution in **564.2342 ms** when executed using a single process. When the number of MPI processes increased to two and three processes across multiple laptops, the execution time increased to **4500.2451 ms**, **8331.9176 ms**, and **8691.5135 ms**, respectively. This increase was mainly caused by communication overhead introduced by MPI, especially when data had to be exchanged between different laptops through the network.

For the dataset size of **10,000,000 records**, the total runtime increased substantially. The recorded execution times were **74268.9270 ms** and **78565.4887 ms** for two distributed executions using three MPI processes. The increase in execution time reflects the higher computational workload together with the additional communication cost required for processing a much larger dataset.

Overall, the results indicate that dataset size has a direct impact on execution time. Larger datasets require more processing time and greater communication between MPI processes, particularly in a distributed computing environment.

## 6.3 Performance of Individual Analytical Tasks

The runtime of each analytical task was analysed to determine which component contributed the most to the overall execution time.

### Basic Statistics

Basic Statistics required relatively little processing time. For the dataset containing **1,000,000 records**, the execution time ranged from **9.40 ms** to **100.70 ms**. For the

**10,000,000-record** dataset, the execution time ranged from **119.78 ms** to **164.08 ms**. This indicates that calculating the mean, minimum, maximum, and variance is computationally inexpensive compared to other analytical tasks.

### **Histogram Generation**

Histogram generation required only a small amount of processing time. The execution time remained below **31 ms** even for the largest dataset. This shows that histogram generation scales efficiently as the dataset size increases.

### **Sorting**

Sorting was consistently the most computationally expensive operation throughout the experiment.

For the dataset size of **1,000,000 records**, sorting required between **397.86 ms** and **3833.82 ms**.

For the dataset size of **10,000,000 records**, sorting required between **34965.25 ms** and **38212.52 ms**.

Sorting accounted for the largest percentage of the total runtime because every process first sorted its own local data before the sorted data were gathered and merged by the master process. This gathering stage introduced additional processing overhead.

### **Pearson Correlation**

Pearson Correlation required a moderate amount of execution time. Most executions completed between **14 ms** and **211 ms**. Compared to sorting, Pearson Correlation consumed only a small portion of the total runtime.

### **Moving Average**

Moving Average was the fastest analytical task. The execution time remained below **4 ms** for all experiments because only the first and last moving averages were calculated.

### **Outlier Detection**

Outlier Detection also required relatively little processing time. For the dataset size of **10,000,000 records**, the execution time remained below **90 ms**, which is significantly lower than the sorting process.

## **6.4 Distributed MPI Performance**

The MPI program was successfully executed using three laptops connected to the same network. The experiment demonstrated that distributed execution was possible after configuring the network and establishing communication between the MPI processes.

Although the distributed implementation successfully completed all analytical tasks, the overall runtime increased compared with the single-process execution. This occurred

because additional communication was required to distribute the dataset, synchronize processes, and gather the final results from each laptop. Network latency and communication overhead therefore became significant factors affecting overall performance.

The experiment also showed that sorting remained the dominant contributor to the total runtime, even when the workload was distributed among multiple processes.

## 6.5 Discussion of Findings

The experimental results show that the execution time increases as the dataset size becomes larger. Processing 10,000,000 records required substantially more execution time than processing 1,000,000 records, demonstrating the increased computational complexity associated with larger datasets.

Among all analytical tasks, sorting consistently consumed the highest execution time. For the largest dataset, sorting alone required more than 34,000 ms, accounting for nearly half of the total runtime. This indicates that sorting is the primary performance bottleneck of the statistical analytics pipeline.

The remaining analytical tasks, including Basic Statistics, Histogram Generation, Pearson Correlation, Moving Average, and Outlier Detection, required comparatively small execution times and contributed only a small percentage of the overall runtime.

Finally, the distributed MPI experiment confirmed that the program could execute successfully across multiple laptops. However, communication between processing nodes introduced additional overhead, resulting in longer execution times compared to single-process execution. This demonstrates that MPI performance depends not only on computational workload but also on communication efficiency, network latency, and workload distribution across participating nodes.

## 7.0 CHALLENGES AND LESSONS LEARNED

| Challenge                       | Solution / Lesson Learned                                                                                                  |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| MPI overhead for small datasets | Small datasets do not provide enough work to offset MPI communication overhead.<br>MPI is more useful for larger datasets. |

|                              |                                                                                                                                                                                                                                                                         |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| One laptop could not connect | The group planned to use 4 laptops, but one laptop could not connect because it was using a different router or network. Only 3 laptops were successfully connected. This showed that all devices must be connected to the same network before running distributed MPI. |
| Network configuration issue  | The group used ipconfig to check IP addresses and confirm that the laptops were connected under the same network range.                                                                                                                                                 |
| Uneven workload distribution | sendCounts and displacements were used with MPI_Scatterv so that records can be divided correctly even when the dataset size is not perfectly divisible by the number of processes.                                                                                     |
| Combining partial results    | MPI_Reduce was used to combine global sums, histogram counts, minimum, maximum, variance, and outlier counts.                                                                                                                                                           |
| Sorting bottleneck           | Local sorting improves part of the work, but final MPI_Gatherv and global sorting at Rank 0 still create overhead. A fully parallel sorting algorithm would be better.                                                                                                  |
| Large memory usage           | Large datasets require more memory, especially when Rank 0 stores the full generated data and gathers sorted data. Future work should reduce memory concentration at Rank 0.                                                                                            |
| Distributed MPI overhead     | Running MPI across laptops is slower than local MPI because data must be transferred through the network. A stable wired network may improve performance.                                                                                                               |

The group planned originally to connect 4 laptops to test MPI for our experiment. One of the laptops did not successfully connect to the MPI network because it was connected to a different router. This resulted in the successful connectivity of 3 of the laptops being used in our experiment. This demonstrated that successful execution of MPI across multiple devices is heavily dependent on having a properly configured network. All the laptops need to be connected to the same router or have the same network; all the laptops need to have the appropriate IP addresses, all the laptops must have allowed access through their firewalls, and all the laptops must have the compatible version installed of MPI. This experience has taught the group that network setup should be tested early to successfully conduct the final experiment.

## 8.0 CONCLUSION

In conclusion, this project successfully developed and compared a sequential statistical analytics pipeline and an MPI-based parallel analytics pipeline. The programs performed six analytical tasks basic statistics, histogram generation, sorting, Pearson correlation, moving average calculation, and outlier detection.

The result shows that MPI is not always faster than sequential processing. For the smallest dataset, MPI is slower because of communication and synchronization overhead. However, as the dataset size increases, MPI becomes more effective. The best result was achieved at 5,000,000 records, where MPI obtained 56.65% runtime improvement, 2.31 speedup, and 57.67% efficiency.

The distributed MPI test using three laptops also showed that MPI can run across multiple devices. However, distributed MPI requires proper network setup, same router connection, firewall permission, and stable communication between laptops.

For future work, the project can be improved by testing different numbers of MPI processes, such as 2, 4, 6, and 8 processes. The program can also be tested using a wired LAN connection to reduce network delay. In addition, a fully parallel sorting algorithm can be implemented to reduce the bottleneck at Rank 0. The program can also be extended to directly analyse the sales.csv file using real sales analytics such as total revenue by region, total revenue by product, and average sales price.

## 9.0 INDIVIDUAL CONTRIBUTION TABLE

| Role                    | Member              | Primary Responsibility                                                      |
|-------------------------|---------------------|-----------------------------------------------------------------------------|
| Node Master / Architect | NUR HAJARIATUL ASMA | Network configuration, MPI cluster setup, master node logic, job scheduling |
| Algorithm Developer     | MUHAMMAD AMINUDDIN  | Sequential C++ implementation, MPI parallel algorithm design and coding     |
| Data Engineer           | SITI NURAINA        | Dataset preparation, data partitioning strategy, data validation            |
| Performance Analyst     | ZUHAIRI IKHWAN      | Benchmarking framework, metrics collection, performance graphs and analysis |

## 10.0 REFERENCES

- [1] A. Newcastle, “Group Assignment: Distributed Data Analytics Using Message Passing Interface (MPI),” GitHub, 2026. [Online]. Available: <https://github.com/atifnewcastle/mpi-project>
- [2] Microsoft, “Microsoft MPI,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi>
- [3] W. Gropp, “MPI Tutorial,” MPI Tutorial. [Online]. Available: <https://mpitutorial.com>
- [4] Open MPI, “Open MPI Documentation Version 4.1,” Open MPI. [Online]. Available: <https://www.open-mpi.org/doc/v4.1/>
- [5] Wikipedia contributors, “Amdahl’s law,” Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Amdahl%27s\\_law](https://en.wikipedia.org/wiki/Amdahl%27s_law)
- [6] cppreference.com, “Date and time utilities,” cppreference.com. [Online]. Available: <https://en.cppreference.com/w/cpp/chrono>